

LAYER 5: FAILSAFE / VETO LAYER

Horizontal Decomposition

DAO Governance Architecture -- Self-Evolving Quadratic Voting Module

This document presents a rigorous horizontal decomposition of Layer 5 (Failsafe / Veto Layer) from the DAO Governance Architecture. Each module is specified with purpose, state, interfaces, events, invariants, and agentic implementation prompts.

Generated for Agentic Coding IDE Integration

Date: 2026-05-10

Architecture Overview

Layer 5 acts as the ultimate circuit breaker for the self-evolving DAO governance system. It ensures that any automated parameter adjustment made by the Intelligence Layer (Layer 3) can be intercepted, vetoed, and reverted to a safe baseline before causing systemic damage.

The layer is decomposed into seven horizontally-aligned modules, each with a single, well-defined responsibility and strict interaction contracts:

LAYER 5: FAILSAFE / VETO LAYER						
Timelock	Veto	Multisig	Ragequit	Baseline	Audit	Alert
Manager	Registry	Coordinator	Engine	Reverter	Logger	Service

Module Summary:

- 1. Timelock Manager -- Enforces mandatory delay on all parameter adjustments
- 2. Veto Registry -- Validates and records all veto actions (cancellation-only)
- 3. Multisig Coordinator -- Manages distributed steward council and threshold logic
- 4. Ragequit Engine -- Economic exit hatch with pro-rata treasury distribution
- 5. Baseline Reverter -- Atomic restoration to immutable safe-mode parameters
- 6. Audit Logger -- Append-only, hash-chained event log for transparency
- 7. Alert Service -- Off-chain notification system for veto window monitoring

Module 1: Timelock Manager

Purpose: Enforces mandatory delay between adjustment proposal and execution

Core Responsibilities

- Queue adjustments originating from Layer 3 (Intelligence & Optimization Layer)
- Enforce DELAY_PERIOD -- no execution permitted before executeAfter timestamp
- Maintain pending queue with status tracking (Queued / Executed / Cancelled)
- Expose read-only interface for external monitoring

Key State

```
pendingAdjustments: Map<adjustmentId => {  
  params: ParameterSet,  
  queuedAt: uint256,  
  executeAfter: uint256,  
  status: enum {QUEUED, EXECUTED, CANCELLED}  
}>
```

Interface / API

```
queueAdjustment(params: ParameterSet, proof: Layer3Proof)  
  => returns adjustmentId: bytes32  
  
executeAdjustment(adjustmentId: bytes32)  
  => returns bool  
  requires: block.timestamp >= executeAfter  
  requires: status == QUEUED  
  
getPending()  
  => returns Adjustment[]  
  
cancelAdjustment(adjustmentId: bytes32, vetoProof: VetoProof)  
  => callable only by Veto Registry
```

Events

```
event AdjustmentQueued(  
  bytes32 indexed adjustmentId,  
  ParameterSet params,  
  uint256 executeAfter  
);  
  
event AdjustmentExecuted(bytes32 indexed adjustmentId);  
  
event AdjustmentCancelled(  
  bytes32 indexed adjustmentId,  
  string reason  
);
```

Security Invariants

- block.timestamp >= executeAfter is a hard requirement for execution
- Only Layer 3 Intelligence contract can call queueAdjustment()
- Only Veto Registry can trigger cancelAdjustment()

- Cancelled adjustments can never be re-executed

Agent Prompt Hint

"Implement a time-locked escrow for governance parameters. No execution before delay. Immutable queue. The queue must be append-only and its entries must be cryptographically linked to the originating Layer 3 proof."

Module 2: Veto Registry

Purpose: Central registry for all veto actions -- validates eligibility and prevents unauthorized cancellations

Core Responsibilities

- Accept veto submissions from authorized stewards
- Validate steward authority via Multisig Coordinator
- Record veto rationale (as hash) for transparency
- Trigger cancellation in Timelock Manager when threshold is met
- Initiate Baseline Reverter upon successful veto

Key State

```
vetoos: Map<adjustmentId => VetoEntry[]>
  where VetoEntry = {
    steward: address,
    timestamp: uint256,
    rationaleHash: bytes32,
    executed: bool
  }

vetoCountPerAdjustment: Map<adjustmentId => uint256>

stewardVoted: Map<adjustmentId => Map<address => bool>>  // anti-double-vote
```

Interface / API

```
submitVeto(
  adjustmentId: bytes32,
  rationaleHash: bytes32,
  signature: bytes
) => returns vetoId: bytes32
  requires: isSteward(msg.sender)
  requires: !stewardVoted[adjustmentId][msg.sender]

executeVeto(adjustmentId: bytes32)
  => returns bool
  requires: vetoCount >= threshold
  effects: calls timelock.cancelAdjustment()
  effects: calls baselineReverter.triggerRevert()

getVetoStatus(adjustmentId: bytes32)
  => returns VetoStatus {NONE, PENDING, EXECUTED}
```

Events

```
event VetoSubmitted(
  bytes32 indexed vetoId,
  bytes32 indexed adjustmentId,
  address indexed steward
);

event VetoExecuted(
  bytes32 indexed adjustmentId,
```

```
    uint256 vetoCount
);

event BaselineRestored(
    bytes32 indexed adjustmentId,
    bytes32 baselineConfigHash
);
```

Security Invariants

- Veto CANNOT initiate new actions -- it can only cancel queued adjustments
- Double-veto by the same steward on the same adjustment is strictly rejected
- Veto must be submitted and executed before executeAfter timestamp
- Veto execution is idempotent -- once executed, further vetoes are rejected

Agent Prompt Hint

"Build a cancellation-only governance module. It can destroy proposals but never create them. Enforce one-vote-per-steward per adjustment. The module must be strictly subtractive -- no additive capabilities permitted."

Module 3: Multisig Coordinator

Purpose: Manages the distributed multisig of elected stewards

Core Responsibilities

- Track steward enrollment, terms, and expiry
- Configure and enforce signature threshold (e.g., 3-of-7, 5-of-11)
- Verify aggregated threshold signatures for sensitive operations
- Manage steward rotation and renouncement

Key State

```
stewards: Set<address>

threshold: uint256 // e.g., 3 of 7

stewardElectionTerm: Map<address => uint256>
  // expiryTimestamp -- no permanent seats

activeStewards: uint256 // cached count for gas efficiency
```

Interface / API

```
isSteward(addr: address)
  => returns bool
  requires: addr in stewards AND block.timestamp < stewardElectionTerm[addr]

verifyThresholdSignatures(
  adjustmentId: bytes32,
  signatures: bytes[]
) => returns bool
  requires: signatures.length >= threshold
  requires: all signatures valid AND from unique active stewards

rotateSteward(old: address, new: address, electionProof: bytes)
  requires: threshold met via steward votes

updateThreshold(newThreshold: uint256, stewardVotes: bytes[])
  requires: newThreshold > activeStewards / 2
```

Events

```
event StewardAdded(
  address indexed steward,
  uint256 termExpiry
);

event StewardRemoved(address indexed steward);

event ThresholdUpdated(
  uint256 oldThreshold,
  uint256 newThreshold
);
```

Security Invariants

- Threshold must always be > 50% of active stewards (supermajority of quorum)
- Steward terms expire -- automatic revocation at term end (no permanent seats)
- Rotation requires consensus of existing stewards (self-governing council)
- Renounced or expired stewards cannot vote or be counted toward threshold

Agent Prompt Hint

"Implement a rotating council multisig with term limits. No single steward can act alone. Verify threshold before any veto execution. The council must decay -- terms expire automatically to prevent ossification of power."

Module 4: Ragequit Engine

Purpose: Economic failsafe allowing stakeholders to exit with pro-rata treasury share

Core Responsibilities

- Calculate pro-rata exit value based on voice credit holdings vs. total supply
- Burn voice credits and associated governance tokens
- Transfer pro-rata treasury share to exiting identity
- Permanently lock future governance rights for the exited identity

Key State

```
ragequitRequests: Map<identity => {  
  amount: uint256,  
  requestedAt: uint256,  
  processed: bool,  
  treasurySnapshot: uint256  
>  
>  
  
totalExited: uint256  
  
exitedIdentities: Set<bytes32> // permanent burn registry
```

Interface / API

```
initiateRagequit(  
  identityProof: ZKProof,  
  voiceCreditBalance: uint256  
) => returns requestId: bytes32  
  requires: identity not in exitedIdentities  
  requires: no active vote by this identity  
  
processRagequit(requestId: bytes32)  
  => returns payoutAmount: uint256  
  effects: transfers payout from treasury  
  effects: burns voice credits  
  effects: marks identity as EXITED in Layer 1  
  
getProRataShare(voiceCreditBalance: uint256)  
  => returns uint256  
  formula: (balance / totalVoiceCredits) * treasurySnapshot
```

Events

```
event RagequitInitiated(  
  bytes32 indexed identity,  
  bytes32 indexed requestId,  
  uint256 creditBalance  
)  
;  
  
event RagequitProcessed(  
  bytes32 indexed requestId,  
  uint256 payout,  
  uint256 treasurySnapshot
```

```
);
```

Security Invariants

- Identity is permanently burned after ragequit -- no re-entry ever permitted
- Payout calculated from treasury snapshot at time of request (not live treasury)
- Cannot ragequit while identity has an active vote in any open proposal
- Ragequit is irreversible -- no appeal or recovery mechanism

Agent Prompt Hint

"Build an irreversible economic exit hatch. Burn the identity. Calculate fair share from treasury snapshot at request time. No returns allowed. The exit must be atomic: burn, pay, and ban in a single transaction."

Module 5: Baseline Reverter

Purpose: Atomic restoration to safe, static baseline configuration on veto trigger

Core Responsibilities

- Store immutable baseline parameters hardcoded at deployment
- Execute atomic revert of all dynamic parameters upon veto trigger
- Validate baseline integrity (hash verification)
- Emit cryptographic restoration proof

Key State

```
baselineConfig: BaselineParams // immutable, constructor-set
    struct BaselineParams {
        uint256 quorum;
        uint256 votingDuration;
        uint256 creditRegenRate;
        uint256 maxCreditAllocation;
    }

baselineConfigHash: bytes32 // keccak256(abi.encode(baselineConfig))

lastRestoredAt: uint256

isBaselineActive: bool
```

Interface / API

```
getBaselineConfig()
    => returns BaselineParams
    // pure view -- no storage modification

triggerRevert(vetoProof: VetoProof)
    => returns bool
    callable only by: Veto Registry
    effects: overwrites all dynamic params in Layer 2 with baselineConfig
    effects: sets isBaselineActive = true
    effects: sets lastRestoredAt = block.timestamp
    effects: pauses Layer 3 optimization until manual review

isBaselineActive()
    => returns bool
```

Events

```
event BaselineRestored(
    uint256 timestamp,
    bytes32 baselineConfigHash,
    bytes32 triggeredByVetoId
);
```

Security Invariants

- Baseline parameters are immutable -- only activation is triggered, never the values
- Revert is atomic and immediate -- no staged or partial reversion permitted

- Layer 3 Intelligence cannot override or bypass baseline reversion
- Restoration proof must be cryptographically verifiable (hash chain)

Agent Prompt Hint

"Create an immutable safe-mode config. When triggered, overwrite all dynamic params with hardcoded defaults. No Layer 3 override possible. The baseline must be burned into the contract -- not upgradeable, not governable, not touchable."

Module 6: Audit Logger

Purpose: Immutable, append-only logging of all Layer 5 activities

Core Responsibilities

- Log all state transitions across Layer 5 modules
- Maintain cryptographic chaining (each entry hashes previous entry)
- Expose query interface for post-hoc analysis and dispute resolution
- Prevent log tampering via append-only enforcement

Key State

```
logChain: Map<uint256 => {  
  entryHash: bytes32,  
  prevHash: bytes32,  
  timestamp: uint256,  
  eventType: EventType,  
  payload: bytes  
>  
>
```

```
latestLogIndex: uint256
```

EventType enum:

```
{ TIMELOCK_QUEUED, TIMELOCK_EXECUTED, TIMELOCK_CANCELLED,  
  VETO_SUBMITTED, VETO_EXECUTED, BASELINE_RESTORED,  
  RAGEQUIT_INITIATED, RAGEQUIT_PROCESSED,  
  STEWARD_ADDED, STEWARD_REMOVED, THRESHOLD_UPDATED }
```

Interface / API

```
appendLog(  
  eventType: EventType,  
  payload: bytes,  
  caller: address  
) => returns logId: uint256  
  callable only by: Layer 5 internal modules  
  effects: entryHash = keccak256(prevHash || timestamp || eventType || payload)  
  
verifyChainIntegrity()  
  => returns bool  
  // traverses entire chain, verifies hash linkage  
  
getLogsByType(  
  eventType: EventType,  
  from: uint256,  
  to: uint256  
) => returns LogEntry[]
```

Events

```
event LogAppended(  
  uint256 indexed logId,  
  EventType eventType,  
  bytes32 entryHash
```

```
);
```

Security Invariants

- Append-only -- no deletion, modification, or reordering of entries permitted
- Every Layer 5 state change MUST emit a log entry (completeness guarantee)
- Hash chain links each entry to its predecessor -- tampering breaks the chain
- Log entries are permanent -- no pruning or archival rotation

Agent Prompt Hint

"Build an append-only blockchain-like log inside the contract. Every veto, every revert, every ragequit gets hashed and chained. If you can delete a log, you have failed. If you can reorder logs, you have failed."

Module 7: Alert Service

Purpose: Real-time off-chain notification for veto window monitoring

Core Responsibilities

- Monitor AdjustmentQueued events from Timelock Manager
- Push alerts to steward notification channels
- Track veto deadlines and escalate near expiration
- Maintain alert history for accountability

Key State (Off-Chain / Coprocessor)

```
stewardContactRegistry: Map<address => {  
  channel: ChannelType, // WEBHOOK | EMAIL | SMS | PUSH  
  endpoint: string,      // encrypted  
  publicKey: bytes       // for E2E encryption  

```

```
alertHistory: Map<adjustmentId => AlertEntry[]>  
  where AlertEntry = {  
    timestamp: uint256,  
    urgencyLevel: Urgency,  
    channel: ChannelType,  
    delivered: bool  
  }
```

```
Urgency enum: { LOW, MEDIUM, HIGH, CRITICAL }
```

Interface / API

```
registerAlertChannel(  
  stewardAddr: address,  
  encryptedEndpoint: bytes  
) => returns bool  
  requires: msg.sender == stewardAddr  
  
dispatchAlert(  
  adjustmentId: bytes32,  
  urgencyLevel: Urgency  
) => returns bool  
  effects: decrypts endpoint, dispatches via channel adapter  
  effects: records delivery status in alertHistory  
  
getAlertHistory(adjustmentId: bytes32)  
  => returns AlertEntry[]
```

Events

```
event AlertDispatched(  
  bytes32 indexed adjustmentId,  
  address indexed steward,  
  Urgency urgencyLevel,  
  uint256 timestamp  
);
```

Security Invariants

- Alert service cannot influence on-chain contract state -- read-only monitoring
- Steward endpoints encrypted at rest -- no plaintext storage
- Escalation is advisory only -- never auto-veto or auto-execute
- Alert delivery is best-effort; missed alerts do not invalidate veto rights

Agent Prompt Hint

"Build a watcher bot that screams when proposals enter timelock. It can notify but never vote. Integrate with webhook, email, SMS. The bot is a spectator with a megaphone -- not a participant."

Cross-Module Interaction Flow

The following sequence diagram illustrates the complete lifecycle of a parameter adjustment through Layer 5, including the veto and baseline restoration paths.



Happy Path (No Veto)

- Layer 3 queues adjustment -> Timelock Manager enforces delay
- Alert Service notifies stewards of pending adjustment
- DELAY_PERIOD expires without veto -> adjustment executes to Layer 2
- Audit Logger records the complete sequence

Veto Path

- Stewards submit vetoes to Veto Registry during delay window
- Multisig Coordinator verifies threshold signatures are met
- Veto Registry executes veto -> cancels adjustment in Timelock Manager
- Veto Registry triggers Baseline Reverter -> atomic parameter restoration
- Audit Logger records veto, cancellation, and restoration

Ragequit Path (Independent)

- Identity initiates ragequit -> Ragequit Engine calculates pro-rata share
- Treasury snapshot taken at request time (not live treasury)
- Voice credits burned, identity marked EXITED in Layer 1
- Payout transferred, future governance rights permanently revoked

Agentic IDE Work Packages

The following work packages are designed for parallel agent execution in an agentic coding IDE. Each agent owns specific modules with clear acceptance criteria and interface contracts.

Agent	Module(s)	Acceptance Criteria
Agent A	Timelock Manager + Baseline Reverter	All adjustments delayed by DELAY_PERIOD; baseline restores atomically on veto trigger; no bypass paths exist
Agent B	Multisig Coordinator + Veto Registry	Threshold signatures verified via ECDSA aggregation; veto cannot initiate proposals; double-vote prevented
Agent C	Ragequit Engine	Pro-rata math verified against edge cases (zero supply, overflow); identity permanently burned; no re-entry
Agent D	Audit Logger + Alert Service	Hash chain unbroken across 10k entries; all Layer 5 events logged; alerts fire within 5s of queue event
Integration Agent	Cross-module wiring	End-to-end veto flow passes in <3 blocks; ragequit + baseline revert coexist without reentrancy

Interface Contracts Between Agents

- Agent A exposes: queueAdjustment(), executeAdjustment(), cancelAdjustment(), triggerRevert()
- Agent B exposes: submitVeto(), executeVeto(), isSteward(), verifyThresholdSignatures()
- Agent C exposes: initiateRagequit(), processRagequit(), getProRataShare()
- Agent D exposes: appendLog(), verifyChainIntegrity(), dispatchAlert()
- Integration Agent validates all cross-contract calls via integration tests

Critical Implementation Notes for Agents

The following constraints are non-negotiable. Any agent implementation violating these notes must be rejected at review.

1. Veto-Only Constraint

Enforce at the smart contract level that Veto Registry has no `createProposal()` or `executeAdjustment()` methods. If an agent adds initiation capability, it violates Layer 5's core architectural purpose. The Veto Registry must be strictly subtractive -- it can only cancel, never create.

2. Baseline Immutability

Baseline parameters must be constructor-set constants, not storage variables that can be updated via governance. The Baseline Reverter only ACTIVATES these constants -- it never modifies them. Verify immutability via code inspection and attempt mutation in test suite.

3. Identity Burn

Ragequit must interact with Layer 1 (Identity & Sybil-Resistance) to permanently mark the identity as EXITED. This prevents re-registration, Sybil re-entry, or credit re-accumulation. The burn must be atomic with payout.

4. Timelock as Deadline

The veto window is strictly defined as: `executeAfter - block.timestamp`. Once `block.timestamp >= executeAfter`, the adjustment is immutable and MUST execute. There is no emergency pause or extension mechanism.

5. No Treasury Touch

Verify that no Layer 5 module can move treasury funds except Ragequit Engine's pro-rata distribution. Timelock, Veto Registry, Baseline Reverter, and Audit Logger must have zero treasury access. This prevents Layer 5 from becoming an attack vector for treasury drainage.

6. Completeness of Audit Log

Every state-changing function in Layer 5 MUST call Audit Logger before returning. This includes successful operations, reverted operations (if state changed before revert), and all veto actions. Missing a log is a critical bug.

7. Alert Service Isolation

The Alert Service runs off-chain or in a verifiable coprocessor. It must never hold private keys, never submit transactions, and never influence on-chain state. It is a read-only watcher with notification capabilities only.